



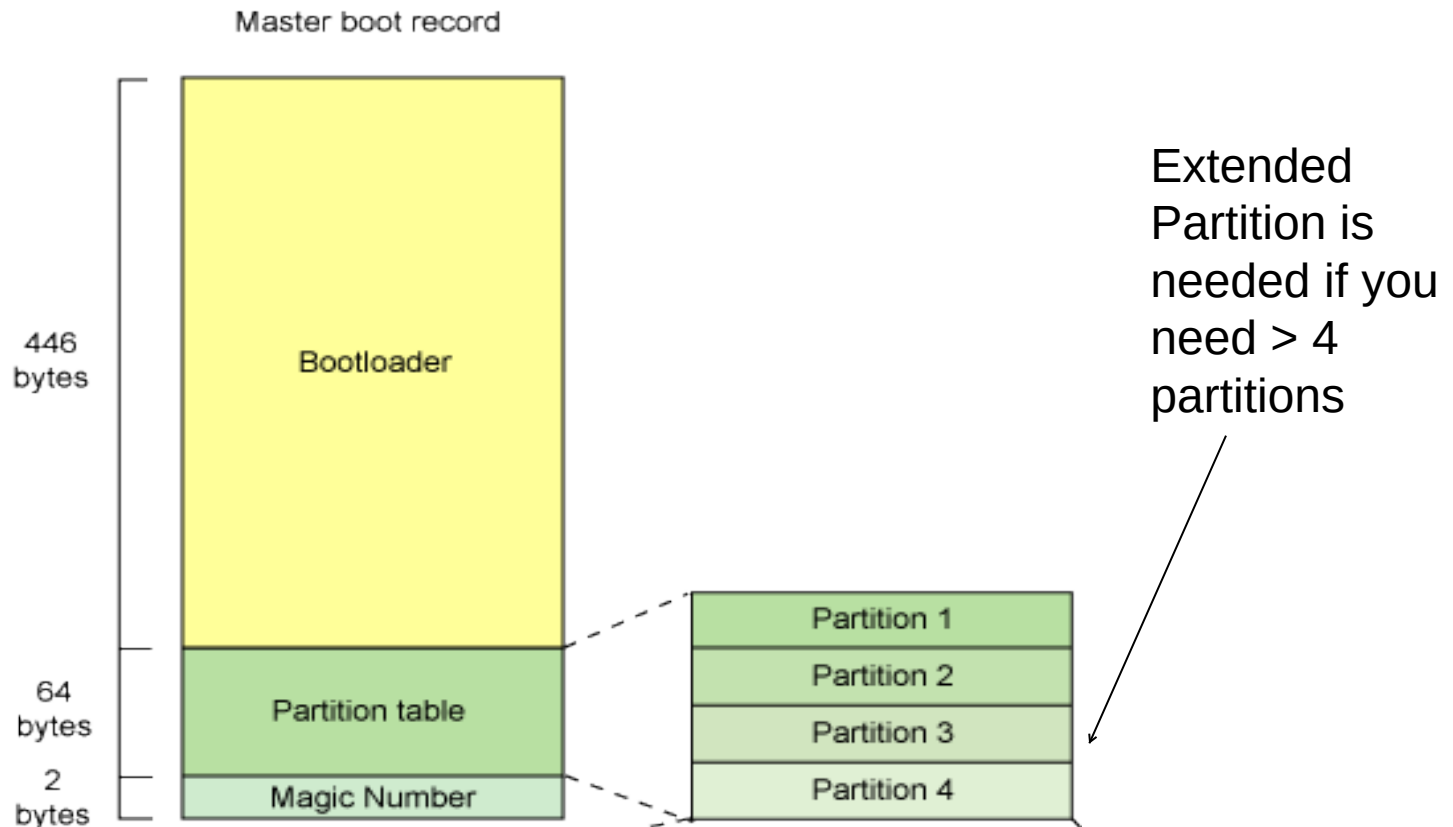
# Process Description and Control

## Chapter 3



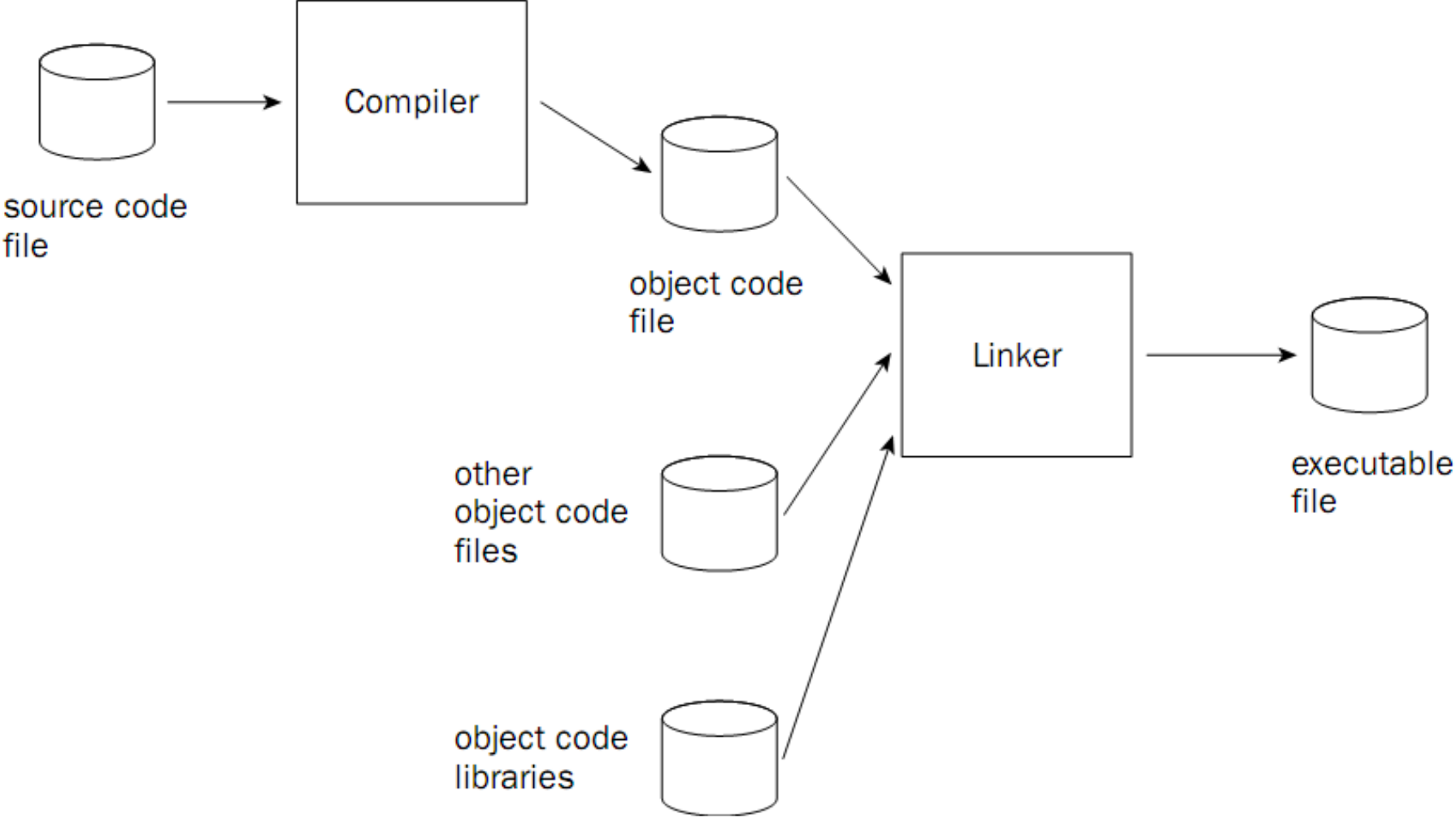
# Boot

- 4 PC powered on, the PC searches for a disk that has a magic number (0xAA55), the 1<sup>st</sup> sector of the disk (master boot record) is read (by BIOS)





# Definitions of Process





# Definitions of Process

- 4 A program in execution
- 4 An instance of a program running on a computer
- 4 The entity that can be assigned to and executed on a processor ← Can be a process or thread
- 4 A unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system instructions



# Definitions of Process

Task Manager

File Options View

Processes Performance App history Startup Users Details Services

| Name                  | PID   | Status  | User name   | CPU | Memory (p... | Th... | Description         |
|-----------------------|-------|---------|-------------|-----|--------------|-------|---------------------|
| ApplicationFrameHo... | 11364 | Running | User        | 00  | 11,368 K     | 4     | Application Fram... |
| atieclxx.exe          | 2920  | Running | SYSTEM      | 00  | 1,364 K      | 2     | AMD External Eve... |
| atiesrxx.exe          | 2228  | Running | SYSTEM      | 00  | 984 K        | 4     | AMD External Eve... |
| audiodg.exe           | 12836 | Running | LOCAL SE... | 00  | 5,156 K      | 5     | Windows Audio ...   |
| Calculator.exe        | 10480 | Running | User        | 00  | 18,520 K     | 10    | Calculator.exe      |
| ChargeService.exe     | 3944  | Running | SYSTEM      | 00  | 1,492 K      | 3     | Super Charger Se... |
| chrome.exe            | 10344 | Running | User        | 00  | 456,628 K    | 31    | Google Chrome       |
| chrome.exe            | 9872  | Running | User        | 00  | 396,152 K    | 18    | Google Chrome       |
| chrome.exe            | 10204 | Running | User        | 00  | 233,568 K    | 34    | Google Chrome       |
| chrome.exe            | 6432  | Running | User        | 00  | 92,096 K     | 16    | Google Chrome       |
| chrome.exe            | 10472 | Running | User        | 00  | 54,508 K     | 14    | Google Chrome       |



# Process

## 4 Consists of three components

- An executable program
- Associated data needed by the program
- Execution context (process state) of the program
  - All information the operating system needs to manage the process, typically store in a **Process Control Block**



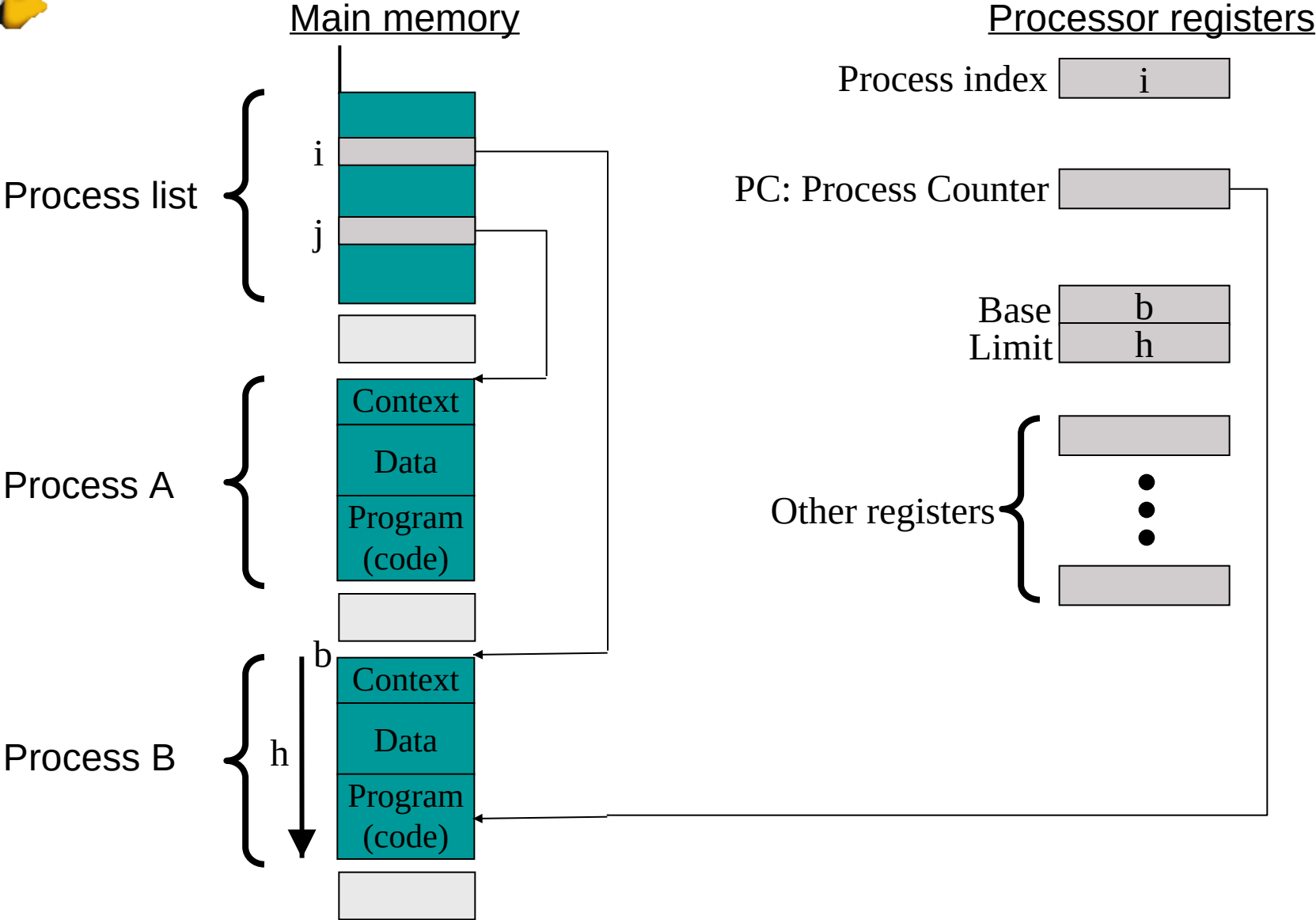
# Process Control Block (PCB)

## 4 Contains the process elements

- **Identifiers:** (process identifier, parent identifier, user identifier)
- **State:** If the process is currently executing, it is in the running state
- **Priority:** Priority level relative to other processes
- **Program Counter:** The address of the next instruction in the program to be executed
- **Memory Pointers:** Includes pointers to the program code and data associated with this process, plus any memory blocks shared with other processes
- **Context Data:** Data that are present in registers in the processor while the process is executing
- **I/O Status Information:** Includes outstanding I/O requests, IO devices assigned to this process, a list of files in use by the process etc.
- **Accounting Information:** Includes the amount of processor time and clock time used, time limits, account numbers and so on.



# Process (Typical Implementation)







# Trace of Process

- 4 Sequence of instruction that execute for a process
- 4 Dispatcher (a program) switches the processor from one process to another



# Trace of Processes

|                        |                        |                        |
|------------------------|------------------------|------------------------|
| 5000                   | 8000                   | 12000                  |
| 5001                   | 8001                   | 12001                  |
| 5002                   | 8002                   | 12002                  |
| 5003                   | 8003                   | 12003                  |
| 5004                   |                        | 12004                  |
| 5005                   |                        | 12005                  |
| 5006                   |                        | 12006                  |
| 5007                   |                        | 12007                  |
| 5008                   |                        | 12008                  |
| 5009                   |                        | 12009                  |
| 5010                   |                        | 12010                  |
| 5011                   |                        | 12011                  |
| (a) Trace of Process A | (b) Trace of Process B | (c) Trace of Process C |

5000 = Starting address of program of Process A  
8000 = Starting address of program of Process B  
12000 = Starting address of program of Process C



# Trace of Processes

Assume that a process execution is limited to 6 instruction cycles, to prevent a single process from monopolizing the processor time

|    |       |                         |    |       |               |
|----|-------|-------------------------|----|-------|---------------|
| 1  | 5000  |                         | 27 | 12004 |               |
| 2  | 5001  |                         | 28 | 12005 |               |
| 3  | 5002  |                         |    |       | -----Time out |
| 4  | 5003  | A                       | 29 | 100   |               |
| 5  | 5004  |                         | 30 | 101   |               |
| 6  | 5005  |                         | 31 | 102   |               |
|    |       | -----Time out           | 32 | 103   |               |
| 7  | 100   | Dispatcher instructions | 33 | 104   |               |
| 8  | 101   |                         | 34 | 105   |               |
| 9  | 102   |                         | 35 | 5006  |               |
| 10 | 103   |                         | 36 | 5007  |               |
| 11 | 104   |                         | 37 | 5008  | A             |
| 12 | 105   |                         | 38 | 5009  |               |
| 13 | 8000  |                         | 39 | 5010  |               |
| 14 | 8001  | B                       | 40 | 5011  |               |
| 15 | 8002  |                         |    |       | -----Time out |
| 16 | 8003  |                         | 41 | 100   |               |
|    |       | -----I/O request        | 42 | 101   |               |
| 17 | 100   |                         | 43 | 102   |               |
| 18 | 101   |                         | 44 | 103   |               |
| 19 | 102   |                         | 45 | 104   |               |
| 20 | 103   |                         | 46 | 105   |               |
| 21 | 104   |                         | 47 | 12006 |               |
| 22 | 105   |                         | 48 | 12007 |               |
| 23 | 12000 |                         | 49 | 12008 | C             |
| 24 | 12001 | C                       | 50 | 12009 |               |
| 25 | 12002 |                         | 51 | 12010 |               |
| 26 | 12003 |                         | 52 | 12011 |               |
|    |       |                         |    |       | -----Time out |

100 = Starting address of dispatcher program

shaded areas indicate execution of dispatcher process;  
first and third columns count instruction cycles;  
second and fourth columns show address of instruction being executed



# Example Execution

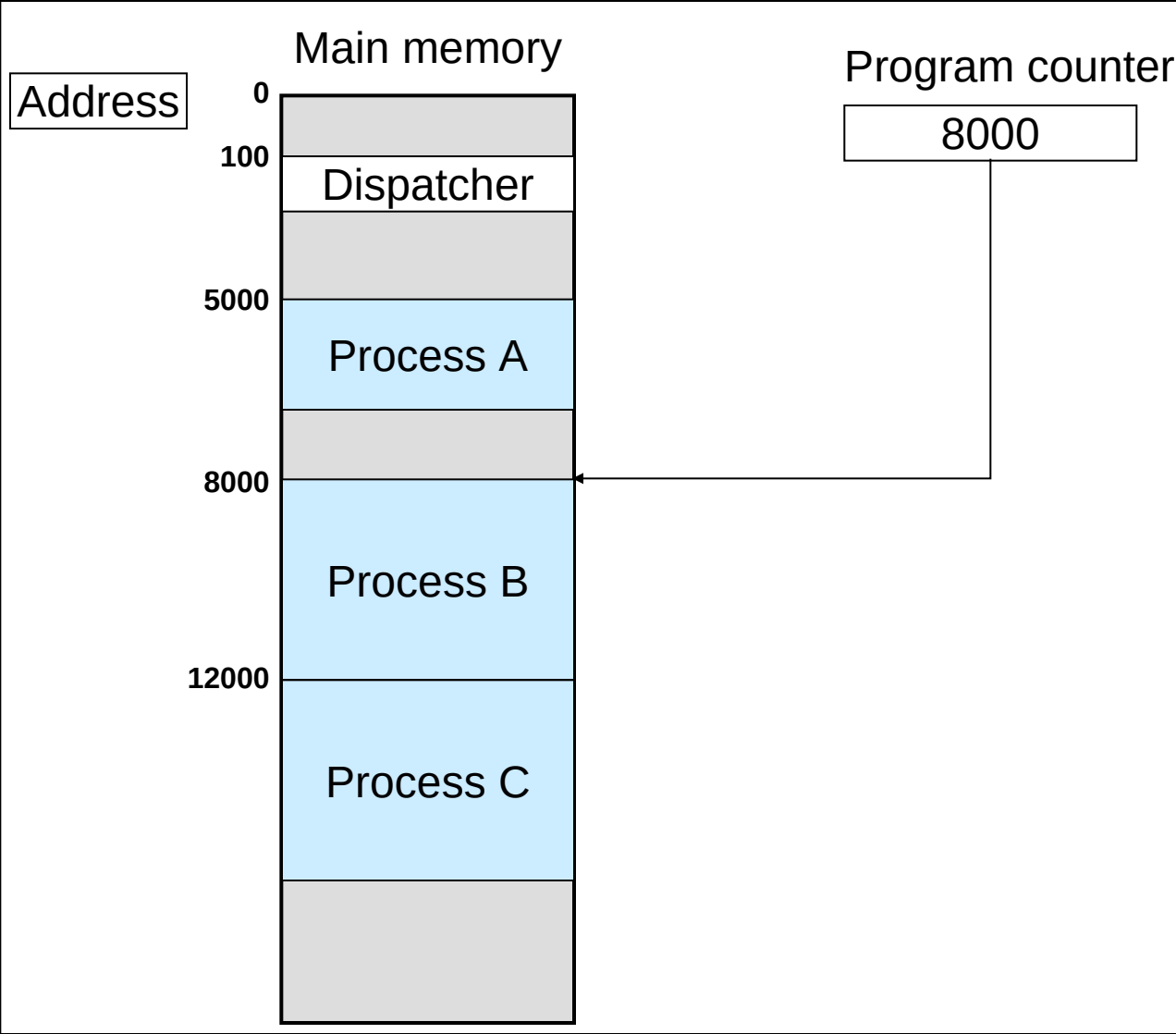


Figure 3.2: Snapshot of example execution (Fig. 3.4) at Instruction Cycle 13



# Process Creation

## ► 4 principal events that cause process creation

|                                    |                                                                                                                                                                                   |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| New batch job                      | The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands. |
| Interactive logon                  | A user at a terminal logs on to the system.                                                                                                                                       |
| Created by OS to provide a service | The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).                             |
| Spawned by existing process        | For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.                                                           |



# Reasons for Process Termination

|                     |                                                                                                                                                                                                                                                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Normal completion   | The process executes an OS service call to indicate that it has completed running.                                                                                                                                                                                                                                                      |
| Time limit exceeded | The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input. |
| Memory unavailable  | The process requires more memory than the system can provide.                                                                                                                                                                                                                                                                           |
| Bounds violation    | The process tries to access a memory location that it is not allowed to access.                                                                                                                                                                                                                                                         |
| Protection error    | The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.                                                                                                                                                                 |
| Arithmetic error    | The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate.                                                                                                                                                                                               |



# Reasons for Process Termination

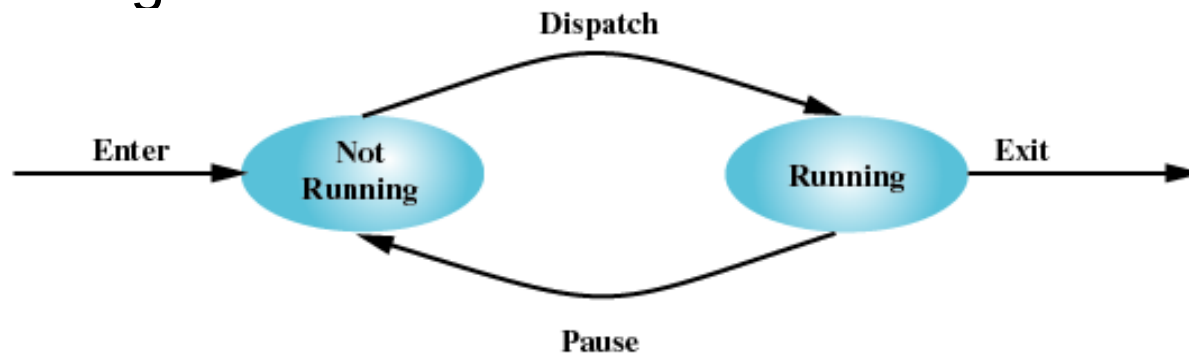
|                             |                                                                                                                                                                                                                                                                            |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Time overrun                | The process has waited longer than a specified maximum for a certain event to occur.                                                                                                                                                                                       |
| I/O failure                 | An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer). |
| Invalid instruction         | The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).                                                                                                                               |
| Privileged instruction      | The process attempts to use an instruction reserved for the operating system.                                                                                                                                                                                              |
| Data misuse                 | A piece of data is of the wrong type or is not initialized.                                                                                                                                                                                                                |
| Operator or OS intervention | For some reason, the operator or the operating system has terminated the process (for example, if a deadlock exists).                                                                                                                                                      |
| Parent termination          | When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.                                                                                                                                                            |
| Parent request              | A parent process typically has the authority to terminate any of its offspring.                                                                                                                                                                                            |



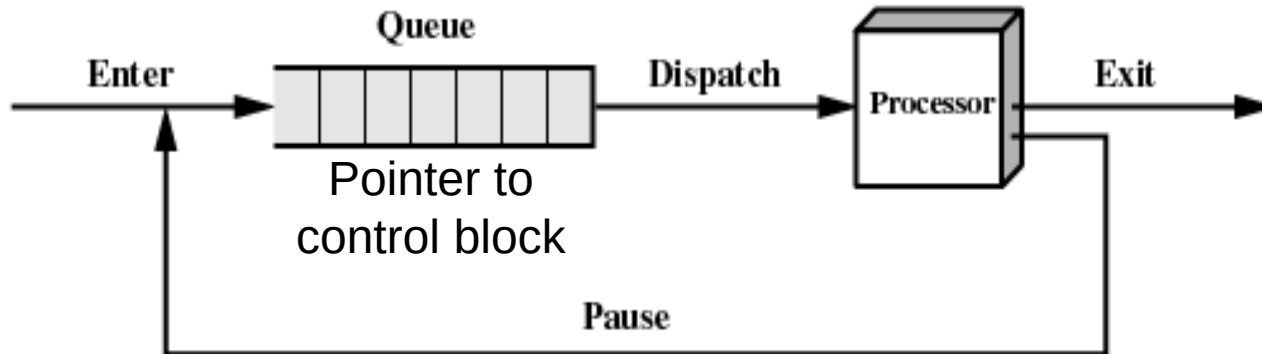
# Two-State Process Model

4 Process may be in one of two states

- Running
- Not-running



(a) State transition diagram



(b) Queuing diagram





# Processes

- 4 Not-running processes are ready to execute but others are blocked (waiting for I/O)
- 4 Using a single queue, dispatcher cannot just select the process that has been in the queue the longest because it may be blocked
  - Scan the list looking for process that is not blocked and has been in the queue the longest.
- 4 Split the *Not Running* state into *Ready* and *Blocked*.

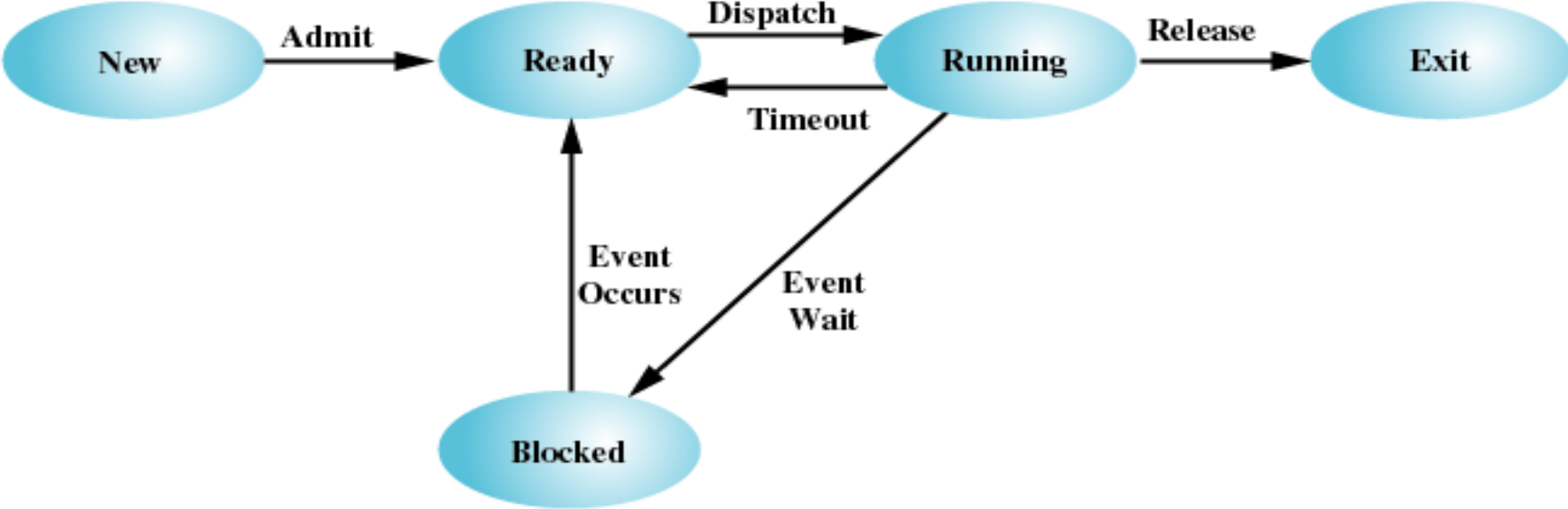


# A Five-State Model

- 4 **New** – a newly created process but has not been admitted to the pool of executable processes, has not loaded into main memory.
- 4 **Ready**
- 4 **Running**
- 4 **Blocked** – a process that cannot execute until some events occurs (e.g. completion of I/O an operation)
- 4 **Exit** – a process that has been released from the pool of executable processes

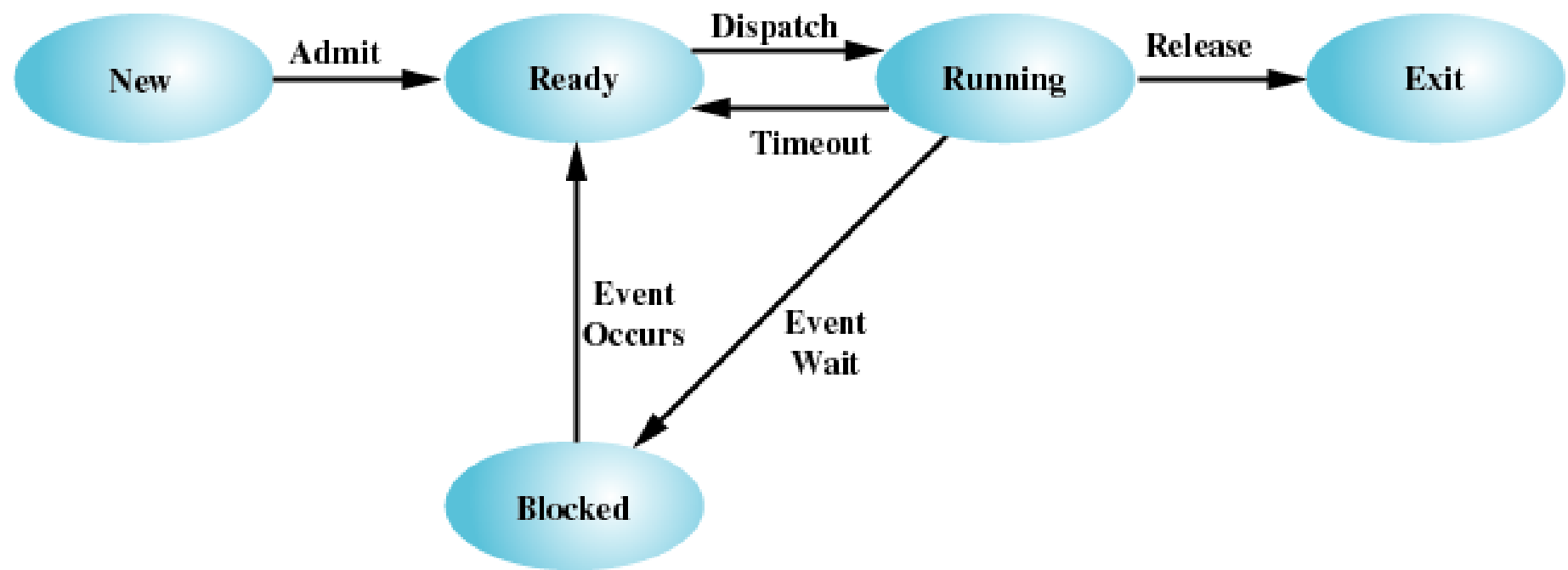


# Five-State Process Model





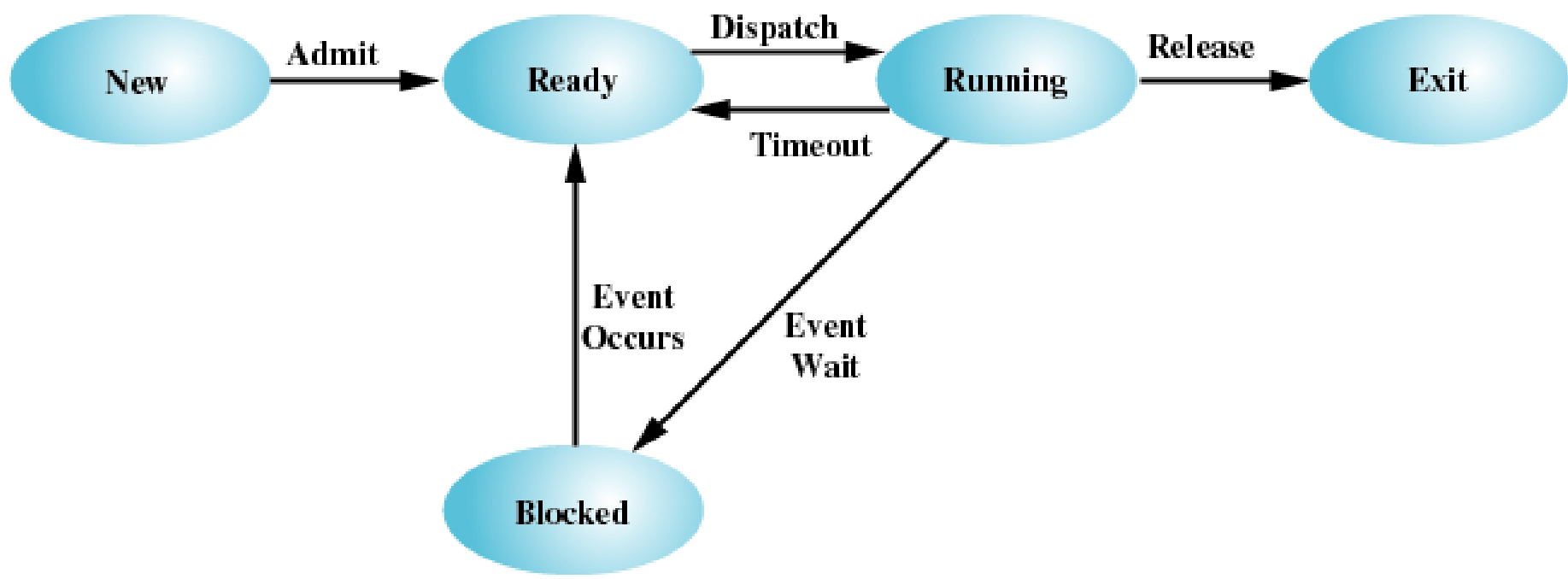
# State Transition



- **New → Ready**: when the OS is prepared to take on additional process. Most systems limit the number of existing processes or the virtual memory committed to existing processes (performance issue)



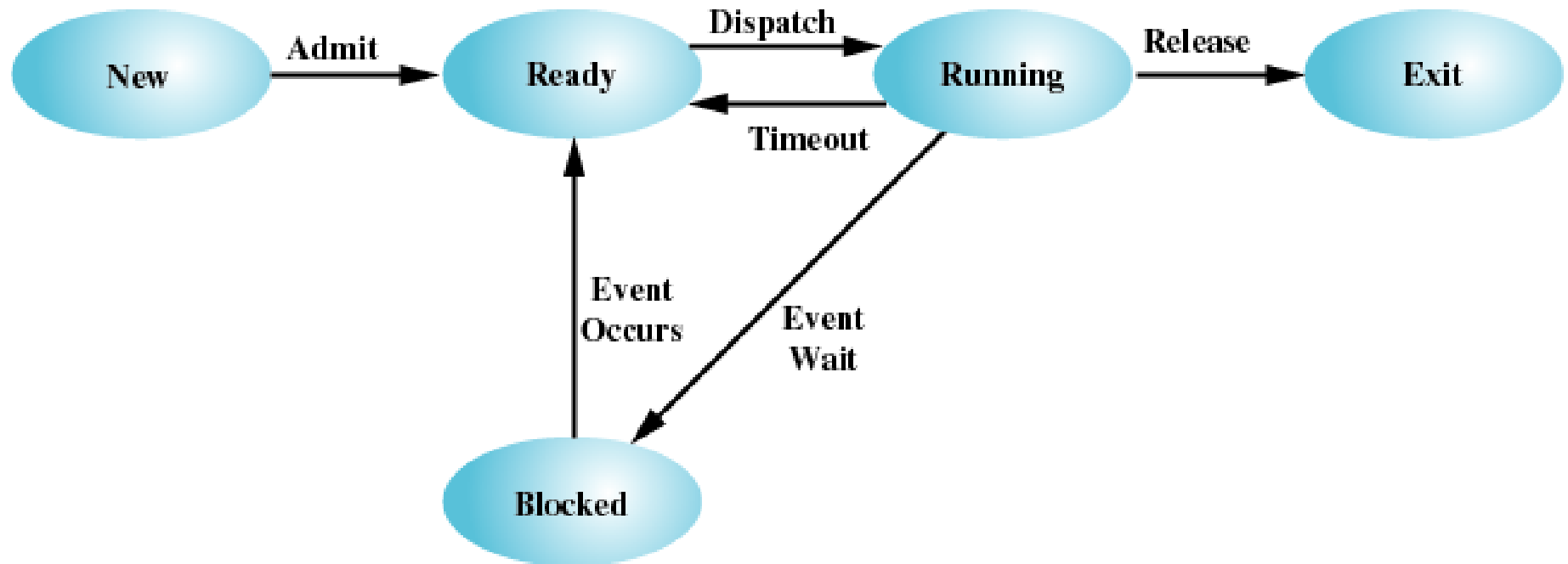
# State Transition



- ▶ **Ready → Running**: The OS will choose 1 of the processes in Ready state when it needs to select new process.
- ▶ **Running → Ready**: the running process has reached the maximum time slice allowed.



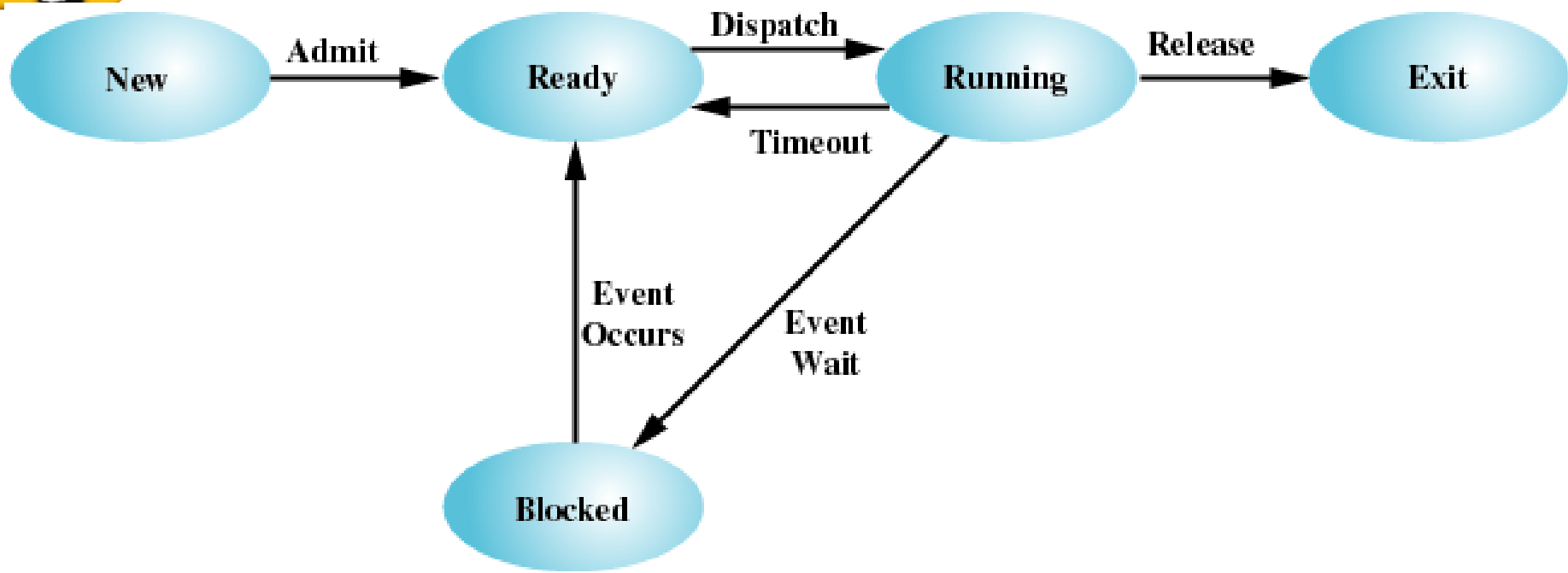
# State Transition



- ▶ **Running → Exit:** The currently running process is terminated by OS when it has completed or it is aborts.
- ▶ **Running → Blocked:** when a process request something for which it must wait. A request to OS is usually in form of a system service call.



# State Transition



- ▶ **Blocked → Ready**: when the event for which it has been waiting has occurs.
- ▶ **Ready → Exit, Blocked → Exit** (not shown in diagram): when a parent process terminates a child process.



# Process States

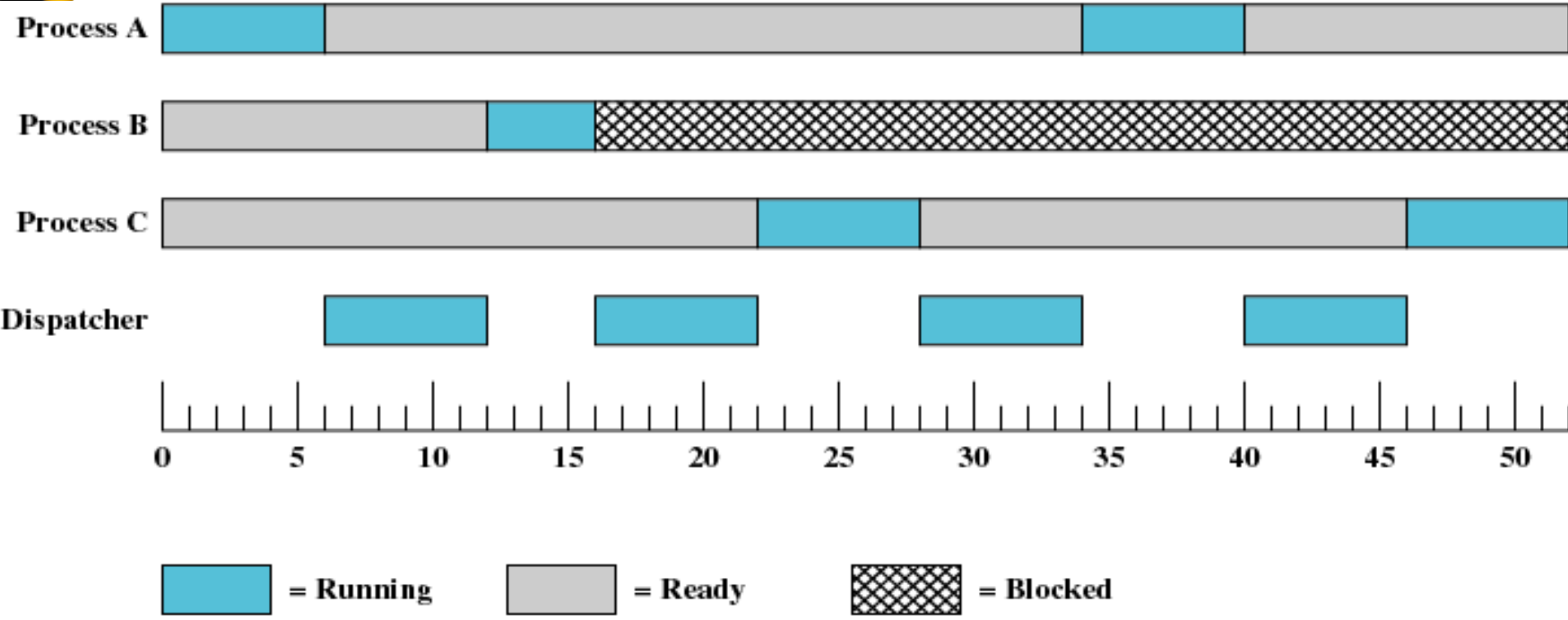
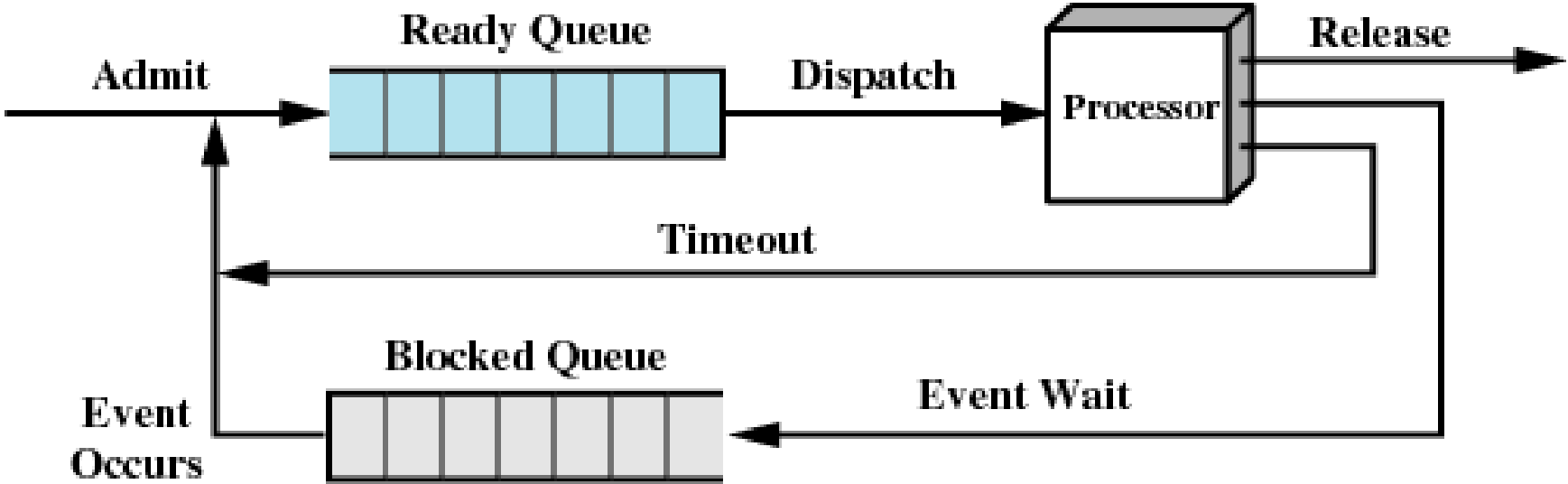


Figure 3.7 Process States for Trace of Figure 3.4





# Using Two Queues

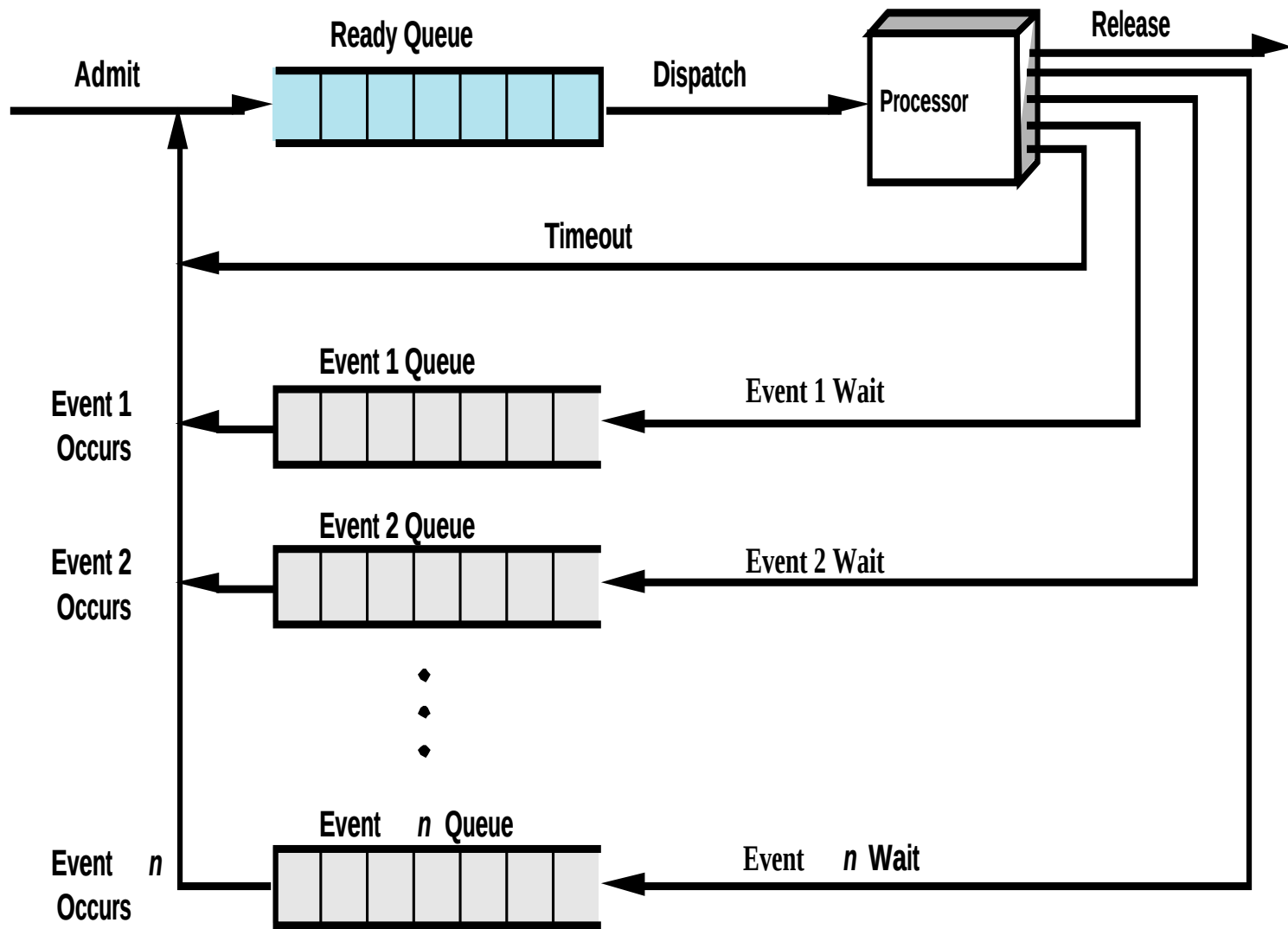


(a) Single blocked queue

- ▶ Using a single blocked queue, when an event occurs, the OS must scan the entire blocked queue, looking for the those processes waiting on that event → inefficient to large OS



# Multiple Blocked Queues



(b) Multiple blocked queues



# Exercise

- 4 Process A is a new process and the execution cycles are as below:
1. Process A executes
  2. Time out (clock interrupt)
  3. Process A continues to execute
  4. Process A waiting for I/O operation
  5. Process A continues to execute
  6. Process A terminate
- 4 State the changes of process state based on 5-state model.



# Exercise (Answer)

4 Process A is a new process and the execution cycles are as below:

1. Process A executes
2. Time out (clock interrupt)
3. Process A continues to execute
4. Process A waiting for I/O operation
5. Process A continues to execute
6. Process A terminate

4 The changes of process state based on 5-state model:

1. New  $\rightarrow$  Ready  $\rightarrow$  Running
2. Running  $\rightarrow$  Ready
3. Ready  $\rightarrow$  Running
4. Running  $\rightarrow$  Blocked
5. Blocked  $\rightarrow$  Ready  $\rightarrow$  Running
6. Running  $\rightarrow$  Exit

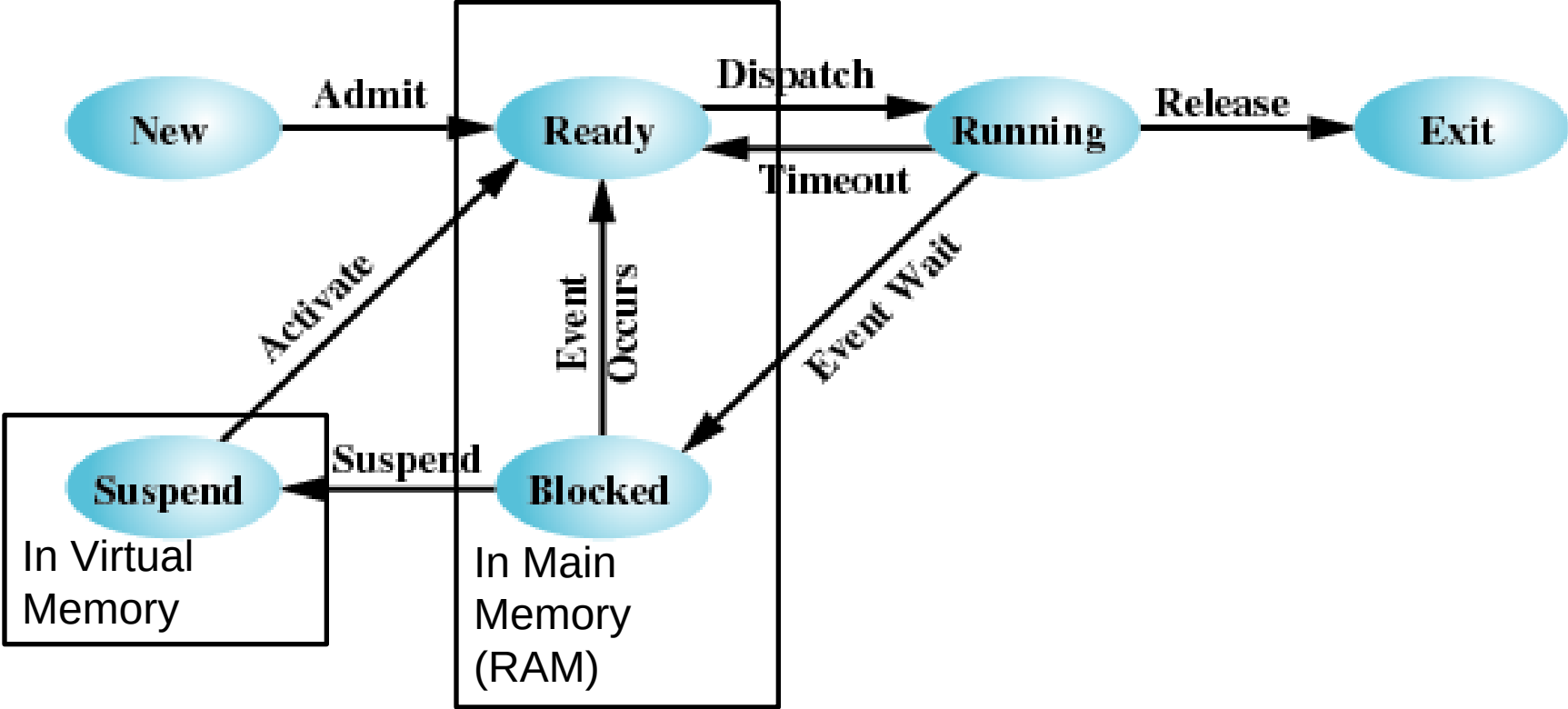


# Suspended Processes

- 4 Processor is faster than I/O so all processes could be waiting for I/O, the processor could be idle most of the time if these processes are stored in memory.
- 4 Swap these processes to disk to free up more memory (swapping), when there are no ready processes in main memory.
- 4 Blocked state becomes suspend state when swapped to disk



# One Suspend State





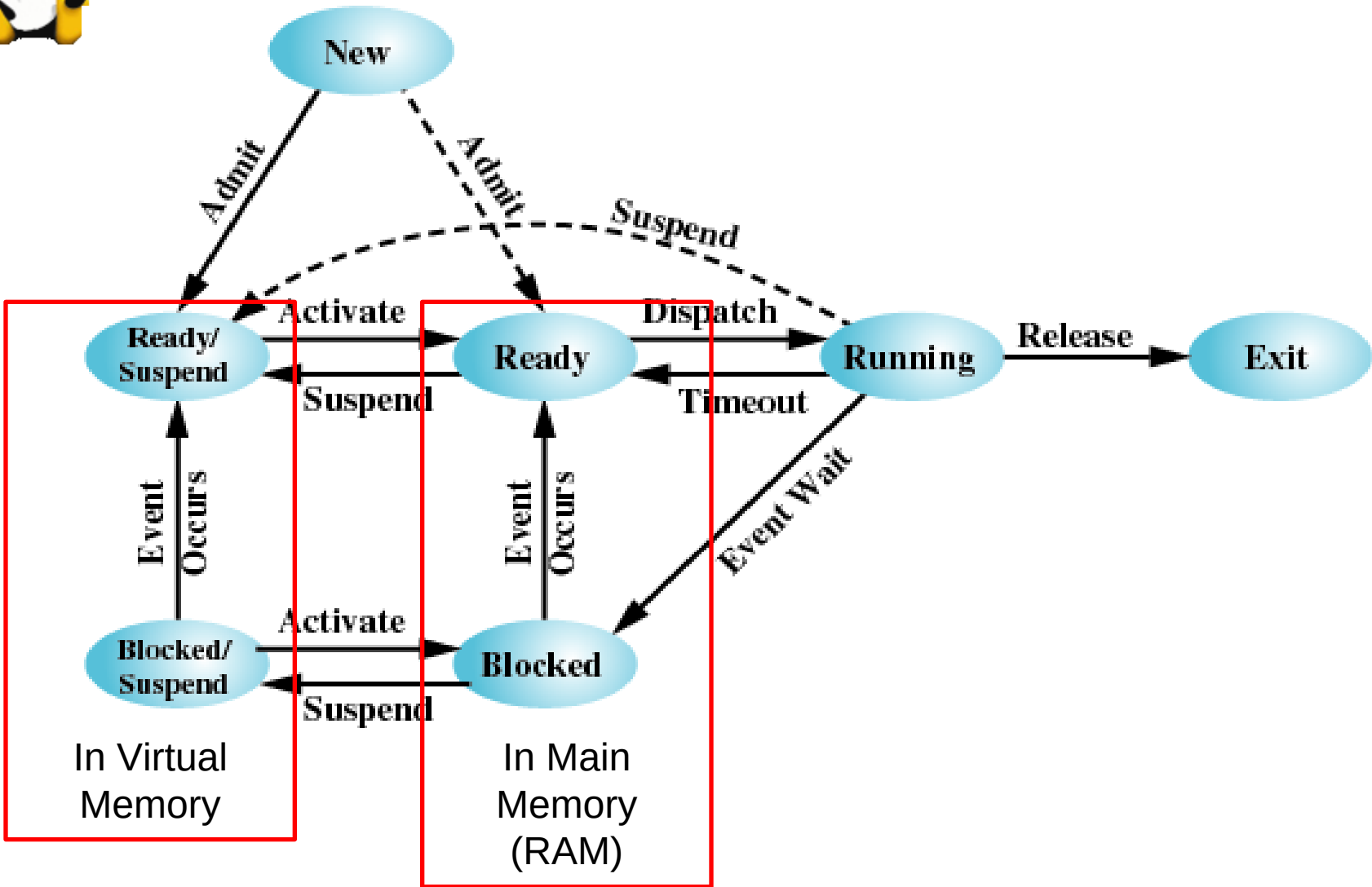
# Suspended Processes

## 4 Two new states

- Blocked/Suspend: the process is in secondary storage, waiting an event.
- Ready/Suspend: the process is in secondary storage, but ready for execution as soon as it is loaded into main memory.



# Two Suspend States



..... Possible but not necessary transition





# New State Transitions

## 4 Blocked → Blocked/Suspend

- When there are no ready processes, at least one process is swapped out.
- If there are ready processes, but the currently running process or a ready process requires more main memory, at least one process still need to be swapped out.

## 4 Blocked/Suspend → Ready/Suspend: when the event for which a process has been waiting occurs.



# New State Transitions

## 4 Ready/Suspend → Ready

- When there are no ready processes in main memory or a ready/suspend process has higher priority than other ready processes.

## 4 Ready → Ready/Suspend:

- if that is the only way to free up a large block of main memory.
- OS may suspend lower priority ready process rather than a higher priority blocked process



# Reasons for Process Suspension

|                          |                                                                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Swapping                 | The operating system needs to release sufficient main memory to bring in a process that is ready to execute.                                                     |
| Other OS reason          | The operating system may suspend a background or utility process or a process that is suspected of causing a problem.                                            |
| Interactive user request | A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.                                         |
| Timing                   | A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.         |
| Parent process request   | A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendents. |



# Exercise

1. A new process, process Q, is now ready to execute and is waiting in the queue.
2. However, process Q is swapped out from main memory to get more memory space for the currently running process.
3. When the currently process terminates, process Q is selected to execute.
4. Process Q issues an I/O request.
5. When the I/O request is done, process Q continues its execution.
6. Timer interrupt occurs after a while.
7. Finally, process Q is terminated by user.

List the state transitions of process Q based on 7-state model.



# Exercise

1. A new process, process Q, is now ready to execute and is waiting in the queue.
  2. However, process Q is swapped out from main memory to get more memory space for the currently running process.
  3. When the currently process terminates, process Q is selected to execute.
  4. Process Q issues an I/O request.
  5. When the I/O request is done, process Q continues its execution.
  6. Timer interrupt occurs after a while.
  7. Finally, process Q is terminated by user.
- 
- 4 The state transitions of process Q based on 7-state model:
    1. New  $\rightarrow$  Ready
    2. Ready  $\rightarrow$  Ready/Suspend
    3. Ready/Suspend  $\rightarrow$  Ready  $\rightarrow$  Running
    4. Running  $\rightarrow$  Blocked
    5. Blocked  $\rightarrow$  Ready  $\rightarrow$  Running
    6. Running  $\rightarrow$  Ready
    7. Ready  $\rightarrow$  Exit



# Operating System Control Structures

4 Manage processes and resources: information about the current status of each process and resource needed.

- Tables are constructed for each entity the operating system manages
- Types of tables:
  - Memory
  - I/O
  - File
  - Process



# Operating System Control Structures

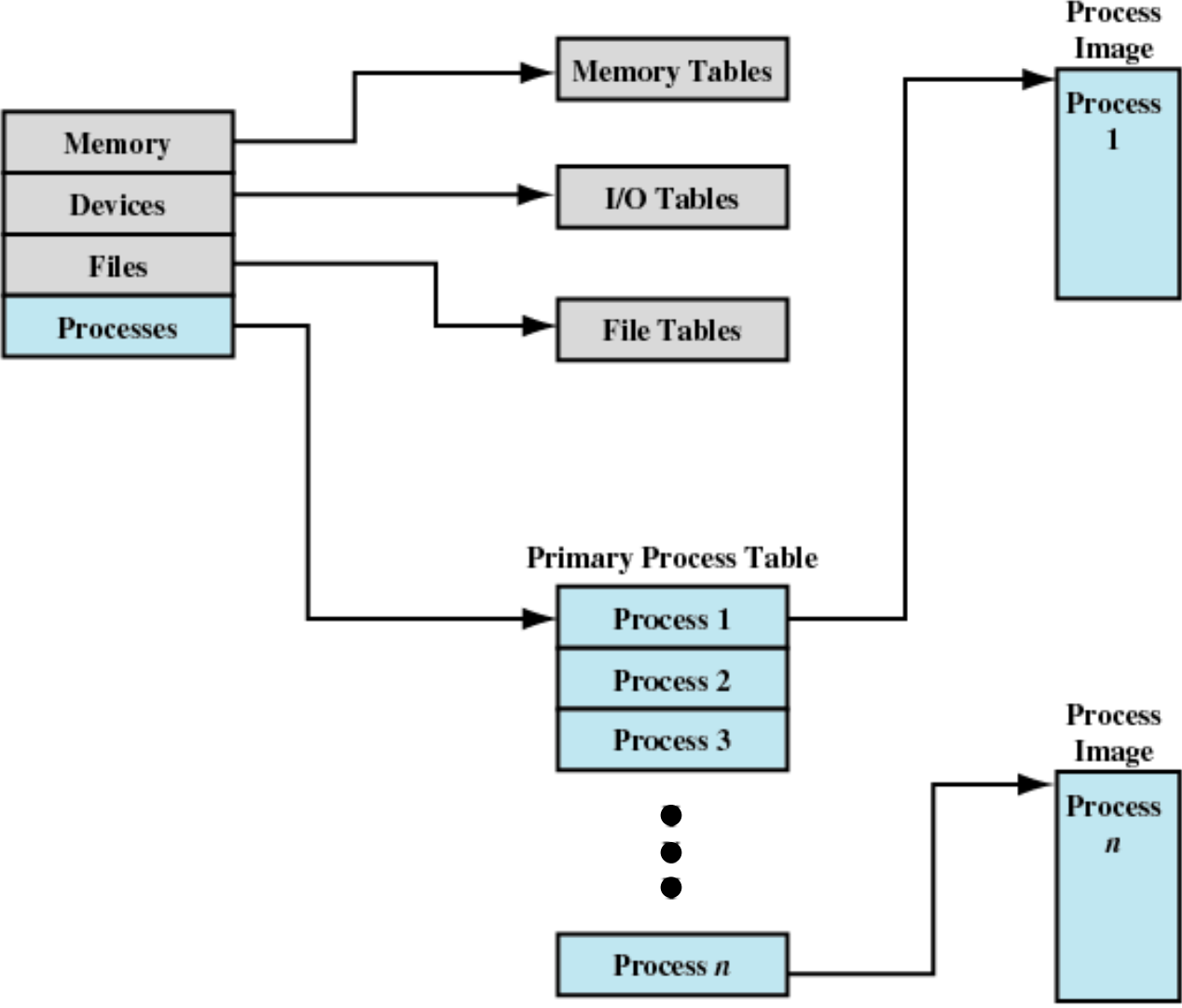


Figure 3.11 General Structure of Operating System Control Tables



# Memory Tables

4 Keep track main and secondary memory.

4 Information in tables:

- Allocation of main memory to processes
- Allocation of secondary memory to processes
- Protection attributes for access to shared memory regions
- Information needed to manage virtual memory





# I/O Tables

## 4 Information in tables:

- I/O device may be available or assigned
- Status of I/O operation (if I/O operation is in progress)
- Location in main memory being used as the source or destination of the I/O transfer



# File Tables

4 Provide the following information

- Existence of files
- Location of files on secondary memory
- Current status of files
- Other attributes

4 Sometimes the information is maintained by a file management system, not the OS



# Process Table

4 Where process is located

4 Information about process:

- Program – the program to be executed
- Data – the modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.
- Stack – each process as 1 or more Last-In-First-Out (LIFO) system stacks associated, used to store parameters & calling addresses for procedure & system calls
- Attributes used by OS (PCB)

4 All the information refer as process image.



# Modes of Execution

## 4 User mode

- Less-privileged mode
- User programs typically execute in this mode

## 4 System mode, control mode, or kernel mode: more-privileged mode

## 4 Mode switch may occur without changing the running state of a process, thus involve little overhead.



# Process Creation

1. Assign a unique process identifier to the new process
2. Allocate space for the process
3. Initialize process control block
4. Set up appropriate linkages
  - Example: add new process to linked list used for scheduling queue
5. Create or expand other data structures
  - Example: maintain an accounting file



# When to Switch a Process

## 4 Clock interrupt

- process has executed for the maximum allowable time slice. The OS will switch the process to **Ready** state.

## 4 I/O interrupt

- If I/O event is accomplished, process that is waiting for that event will be switched to **Ready** state.

## 4 Memory fault

- If the memory address is in virtual memory (which is in secondary memory), the OS must bring in the block of memory to the main memory



# When to Switch a Process

## 4 Trap

- error or exception occurred
- may cause process to be moved to **Exit** state and process switch will occur

## 4 Supervisor call

- such as file open (the execution will be transferred to corresponding routine, which is part of operating system code). It will place the user process in the **Blocked** state.



# Steps in Process Switching

1. Save context of processor including program counter and other registers
2. Update the process control block of the process that is currently running (change process state, update relevant fields)
3. Move process control block to appropriate queue: ready; blocked; ready/suspend
4. Select another process for execution
5. Update the process control block of the process selected
6. Update memory-management data structures
7. Restore context of the selected process





The End